

Lezione 5 - Gerarchia di memoria

GERARCHIA DI MEMORIA

◆ Latenza della memoria

Le operazioni della CPU sono molto veloci, ma gli accessi alla memoria sono molto più lenti.

👉 esempi tipici:

- operazioni aritmetiche → molto veloci
- cache → veloce
- memoria principale → molto lenta

👉 quindi:

✳ il tempo di accesso ai dati può dominare il tempo totale di esecuzione

◆ Effetto pratico

Consideriamo un programma che:

- legge un valore dalla memoria
- lo modifica
- lo riscrive

👉 anche se la CPU è veloce, bisogna aspettare la memoria

👉 quindi il tempo totale dipende soprattutto dagli accessi alla memoria

◆ Problema

Idealmente vorremmo:

👉 avere tutti i dati nei registri (velocissimi)

👉 ma:

- i registri sono pochi
- la memoria è grande ma lenta

◆ Soluzione

Si introduce una **gerarchia di memoria**:

- registri → velocissimi, molto piccoli
- cache → veloce, dimensione intermedia
- memoria principale → lenta, molto grande

◆ Idea fondamentale

L'idea è:

- 👉 tenere i dati più usati vicino alla CPU
- 👉 e quelli meno usati più lontano

◆ Conclusione

La gerarchia di memoria serve per:

- ridurre i tempi di accesso
- migliorare le prestazioni

✳ perché la memoria è il vero collo di bottiglia dei programmi

📌 CACHE

◆ Cos'è la cache

La cache è una memoria più veloce della memoria principale, ma più piccola.

👉 si trova tra CPU e memoria principale

👉 serve per memorizzare temporaneamente i dati usati di recente

◆ Come funziona

Quando la CPU accede a un dato:

1. controlla prima nella cache
2. se il dato è presente → accesso veloce
3. se non è presente → lo prende dalla memoria principale

👉 dopo averlo preso:

- il dato viene copiato nella cache
- accessi successivi saranno più veloci

◆ Effetto

Se accediamo più volte allo stesso dato:

👉 il primo accesso è lento

👉 i successivi sono veloci

◆ Cache per dati e istruzioni

La CPU deve accedere a:

- dati (variabili)
- istruzioni (codice)

👉 per questo spesso esistono due cache separate:

- cache dati
- cache istruzioni

◆ Livelli di cache

Non esiste una sola cache, ma più livelli:

- **L1** → molto veloce, molto piccola
- **L2** → più grande, un po' più lenta
- **L3** → ancora più grande, più lenta

👉 idea:

- più è vicina alla CPU → più è veloce
- più è lontana → più è grande ma lenta

◆ Idea generale

La cache serve per:

- ridurre il numero di accessi alla memoria principale
- migliorare le prestazioni

✳ sfruttando il fatto che spesso usiamo gli stessi dati più volte

📌 PRINCIPI DELLA CACHE

◆ Idea

La cache funziona bene solo se il programma accede ai dati in modo “prevedibile”.

👉 questo comportamento è descritto da due principi fondamentali:

- località temporale
- località spaziale

◆ Località temporale

Se accediamo a un dato, è probabile che lo useremo di nuovo a breve.



Esempio:

```
sum += x;  
sum += x;  
sum += x;
```

👉 la variabile `x` viene riutilizzata più volte

👉 quindi:

- il dato rimane in cache
- accessi successivi sono veloci

◆ Località spaziale

Se accediamo a un indirizzo di memoria, è probabile che accederemo anche agli indirizzi vicini.



Esempio:

```
for (int i = 0; i < N; i++)  
{  
    sum += v[i];  
}
```

👉 stiamo accedendo a elementi consecutivi

👉 quindi:

- quando viene caricato `v[i]`
- spesso vengono caricati anche `v[i+1]`, `v[i+2]`, ecc.

◆ Perché è importante

La cache è efficace solo se questi due principi sono rispettati.

👉 se il programma:

- riusa i dati → località temporale
- accede a dati vicini → località spaziale

✳️ allora la cache funziona molto bene

◆ Caso contrario

Se gli accessi sono:

- casuali
- lontani tra loro
- non ripetuti

👉 la cache è poco efficace

◆ Idea generale

La cache sfrutta:

- località temporale → riuso dei dati
- località spaziale → accesso a dati vicini

✳️ questi due principi spiegano quasi tutte le ottimizzazioni legate alla memoria

📌 CACHE LINES

◆ Cos'è una cache line

La cache non carica un singolo byte alla volta.

👉 quando si accede alla memoria, viene caricato un blocco di dati chiamato **cache line**

◆ Dimensione

Una cache line ha dimensione tipica di:

👉 **64 byte**



Esempio:

- un intero (int) = 4 byte
- in una cache line entrano **16 interi**

◆ Come funziona

Quando accediamo a un dato:

- 👉 non viene caricato solo quel dato
- 👉 ma tutta la cache line che lo contiene
- 👉 quindi:

- accedendo a $v[i]$
- vengono caricati anche $v[i+1]$, $v[i+2]$, ecc.

◆ Effetto

Se accediamo a dati consecutivi:

- 👉 il primo accesso è lento
- 👉 i successivi sono “gratis” (già in cache)

◆ Collegamento con località spaziale

La cache line sfrutta direttamente la località spaziale:

👉 dati vicini vengono caricati insieme

◆ Caso negativo

Se accediamo a dati lontani tra loro:

👉 ogni accesso richiede una nuova cache line

👉 quindi:

- molti accessi lenti
- prestazioni peggiori

◆ Idea generale

La cache line è il motivo per cui:

- accedere a dati consecutivi è veloce
- accedere a dati sparsi è lento

✳ è uno dei concetti più importanti per capire la performance della memoria

📌 ASSOCIATIVITÀ DELLA CACHE

◆ Problema

Quando un dato viene caricato in cache, bisogna decidere:

👉 **dove metterlo**

👉 la cache è divisa in posizioni (slot), e ogni dato deve essere mappato in uno di questi slot

◆ Direct mapped (1-way)

Nel caso più semplice:

👉 ogni indirizzo può andare in **una sola posizione** della cache

👉 problema:

- se due dati diversi vogliono lo stesso slot
- uno dei due viene rimosso

👉 conseguenza:

- molti “rimpiazzi”
- cache meno efficiente

◆ Fully associative

Nel caso opposto:

👉 un dato può essere messo **in qualsiasi posizione** della cache

👉 vantaggi:

- meno conflitti
- maggiore probabilità che il dato sia in cache

👉 svantaggi:

- più complessa da implementare
- più lenta

◆ k-way associative

È un compromesso tra i due:

👉 ogni dato può andare in **k possibili posizioni**



Esempio:

- 2-way → 2 possibili slot
- 8-way → 8 possibili slot

👉 vantaggi:

- meno conflitti rispetto al direct mapped
- più semplice del fully associative

◆ Replacement policy

Se tutte le posizioni possibili sono occupate:

👉 bisogna decidere quale dato eliminare

👉 una politica comune è:

- **LRU (Least Recently Used)** → si elimina il dato usato meno di recente

◆ Idea generale

L'associatività è un compromesso tra:

- velocità della cache
- efficacia (hit rate)

✳ serve per ridurre i conflitti tra dati in memoria

📌 DISPOSIZIONE IN MEMORIA

◆ Liste concatenate

Consideriamo una lista concatenata:

```
struct node
{
    int32_t key;
    struct node * next;
};
```

e un codice che scorre la lista:

```
int sum = 0;
struct node * current = head;

while (current != NULL)
{
    sum += current->key;
    current = current->next;
}
```

◆ Problema

Gli elementi della lista:

👉 non sono contigui in memoria

👉 quindi:

- ogni accesso può puntare a una posizione completamente diversa
- la cache non può sfruttare la località spaziale

👉 conseguenza:

- molti accessi alla memoria
- prestazioni basse

◆ Effetto sulla cache

Quando accediamo a un nodo:

👉 viene caricata una cache line

👉 ma:

- usiamo solo una piccola parte (il campo `key`)

- il resto della cache line è spesso inutile

👉 inoltre:

- il nodo successivo potrebbe non essere nella stessa cache line

✳️ quindi ogni passo può richiedere un accesso lento

◆ Versione migliorata

```
struct node
{
    int key[N];
    bool valid[N];
    struct node * next;
};
```

👉 per ogni nodo abbiamo:

- più valori (key[i])
- un array che indica quali sono validi

◆ Effetto

Quando accediamo a un nodo:

👉 viene caricata una intera cache line

👉 quindi:

- più valori sono già disponibili
- riduciamo gli accessi alla memoria

✳️ sfruttiamo molto meglio la cache rispetto alla lista semplice

ALLINEAMENTO

◆ Idea

In molte architetture, i dati in memoria devono rispettare vincoli di allineamento.

Esempio:

- 4 byte → indirizzo multiplo di 4
- 8 byte → indirizzo multiplo di 8

◆ Padding

Per rispettare l'allineamento, il compilatore inserisce spazio vuoto (padding).

Esempio:

```
struct foo
{
    int8_t a;
    int64_t b;
    int8_t c;
    int64_t d;
    int16_t e;
};
```

👉 contiene molto padding

```
struct bar
{
    int8_t a;
    int8_t c;
    int16_t e;
    int64_t b;
    int64_t d;
};
```

👉 riordinando i campi:

- meno padding
- migliore utilizzo della memoria

```
struct __attribute__((packed)) baz
{
    int8_t a;
    int64_t b;
    int8_t c;
    int64_t d;
    int16_t e;
};
```

👉 senza padding

◆ Effetto

- meno memoria ≠ sempre meglio
- accessi non allineati possono essere più lenti



Visualizzazione della memoria:

```
void print_bytes(char * x, int len)
{
    printf("Dimensione: %d bytes\n", len);
    for (int i = 0; i < len; i++) {
        printf("byte %d: %x\n", i, x[i]);
    }
}
```

```
print_bytes((char *)&f, sizeof(struct foo));
print_bytes((char *)&g, sizeof(struct bar));
print_bytes((char *)&h, sizeof(struct baz));
```

👉 questo codice stampa i byte in memoria delle struct

👉 permette di vedere:

- quanto spazio occupano davvero
- dove sono i byte di padding

✳️ rende concreto l'effetto dell'allineamento

◆ Idea generale

L'allineamento è un compromesso tra:

- velocità
- uso della memoria

✳️ spesso conviene lasciare fare al compilatore



AoS vs SoA

◆ Idea

Spesso abbiamo strutture con più campi e dobbiamo iterare su uno solo.

👉 il modo in cui organizziamo i dati in memoria cambia drasticamente le prestazioni

◆ Array of Structures (AoS)



Nel codice:

```
struct foo
{
    int a;
    int b;
    int c;
    int d;
};

struct foo * v = (struct foo *) malloc(N * sizeof(struct foo));

for (int i = 0; i < N; i++)
{
    v[i].a = rand();
}
```

👉 sommiamo solo a :

```
int sum = 0;
for (int i = 0; i < N; i++)
{
    sum += v[i].a;
}
```

◆ Problema

In memoria abbiamo:

a b c d | a b c d | a b c d ...

👉 quando accediamo a a :

- viene caricata tutta la cache line
- ma usiamo solo una piccola parte

👉 quindi:

- sprechiamo cache
- carichiamo dati inutili (b, c, d)

◆ Structure of Arrays (SoA)



Nel codice:

```
struct array_foo
{
    int * a;
    int * b;
    int * c;
    int * d;
};
```

```
struct array_foo x;
```

```
x.a = (int *) malloc(N * sizeof(int));
x.b = (int *) malloc(N * sizeof(int));
x.c = (int *) malloc(N * sizeof(int));
x.d = (int *) malloc(N * sizeof(int));
```

👉 sommiamo solo a :

```
int sum = 0;
for (int i = 0; i < N; i++)
{
    sum += x.a[i];
}
```

◆ Effetto

In memoria abbiamo:

a a a a a a ...

👉 quindi:

- ogni cache line contiene solo dati utili
- sfruttiamo al massimo la cache

◆ Risultato

👉 SoA è più veloce quando:

- accediamo a un solo campo
- iteriamo su molti elementi

◆ Quando usare cosa

👉 AoS (array di struct):

- quando usiamo più campi insieme
- quando lavoriamo su un singolo oggetto

👉 SoA (struct di array):

- quando iteriamo su un solo campo
- quando vogliamo massimizzare l'uso della cache

◆ Idea generale

La disposizione dei dati in memoria è fondamentale:

- AoS → più semplice ma meno efficiente in certi casi
- SoA → più efficiente ma meno leggibile

✳ scegliere la struttura giusta può cambiare drasticamente le prestazioni

◆ Misura delle prestazioni

Nel codice vengono misurati i tempi di esecuzione con `clock()` .

👉 vengono confrontate due versioni:

- **AoS** → array di struct
- **SoA** → struct di array

👉 nel caso AoS:

```
clock_t start = clock();
for (int i = 0; i < N; i++)
{
    sum += v[i].a;
}
clock_t end = clock();
```

👉 nel caso SoA:

```
start = clock();  
for (int i = 0; i < N; i++)  
{  
    sum += x.a[i];  
}  
end = clock();
```

👉 effetto:

- AoS carica anche campi inutili (b , c , d)
- SoA carica quasi solo i dati che servono

✳ il codice mostra sperimentalmente che la disposizione dei dati in memoria cambia le prestazioni

RICERCA BINARIA E CACHE

◆ Idea

La ricerca binaria ha un comportamento interessante rispetto alla cache.

👉 combina:

- accessi lontani tra loro
- accessi ripetuti ad alcune posizioni

◆ Località temporale

Durante la ricerca binaria:

👉 alcuni elementi (soprattutto quelli centrali) vengono visitati molto spesso



Esempio:

- il primo elemento controllato è circa al centro
- viene visitato in tutte le ricerche

👉 quindi:

- buona località temporale
- quei valori restano in cache

◆ Località spaziale

Durante la ricerca:

👉 all'inizio gli accessi sono molto distanti

- saltiamo in posizioni lontane dell'array

👉 quindi:

- scarsa località spaziale
- molti accessi lenti

👉 invece, verso la fine:

- gli intervalli diventano piccoli
- accediamo a elementi vicini

👉 quindi:

- buona località spaziale
- accessi più veloci

◆ Comportamento complessivo

La ricerca binaria ha:

- buona località temporale all'inizio
- buona località spaziale alla fine

👉 ma:

- non sfrutta bene la cache all'inizio
- è inefficiente per array molto grandi

◆ Miglioramento

Possiamo migliorare la disposizione dei dati in memoria.

👉 non è necessario che l'array sia ordinato in modo lineare

👉 possiamo riordinarlo per:

- migliorare la località
- ridurre i cache miss

👉 idea:

- organizzare i dati come un albero in memoria
- così gli accessi diventano più locali

◆ Versioni della ricerca binaria

💻 Versione con branch

```
int binary_search(int * v, int len, int key)
{
    int l = 0;
    int r = len;
    while (l < r)
    {
        int m = (l + r) / 2;
        if (v[m] >= key)
            r = m;
        else
            l = m + 1;
    }
    return (v[l] == key) ? l : -1;
}
```

👉 usa un `if` , quindi introduce un **branch**

👉 vantaggi:

- semplice
- leggibile
- sfrutta branch prediction e speculazione

👉 limite:

- se la predizione sbaglia, rallenta



Versione branchless

```
int binary_search_branchless(int * v, int len, int key)
{
    int l = 0;
    int r = len;
    while (l < r)
    {
        int m = (l + r) / 2;
        int q = v[m] >= key;
        r = q*m + (1-q)*r;
        l = q*l + (1-q)*(m+1);
    }
    return (v[l] == key) ? l : -1;
}
```

👉 elimina l' if usando operazioni aritmetiche

👉 q vale:

- 1 se vero
- 0 se falso

👉 vantaggi:

- evita branch misprediction

👉 limite:

- non sfrutta l'esecuzione speculativa



Versione branchless con prefetch

```

int binary_search_branchless_prefetch(int * v, int len, int key)
{
    int l = 0;
    int r = len;
    while (l < r)
    {
        int m = (l + r) / 2;
        int q = v[m] >= key;

        __builtin_prefetch(&v[(l+m)/2]);
        __builtin_prefetch(&v[(m+r)/2]);

        r = q*m + (1-q)*r;
        l = q*l + (1-q)*(m+1);
    }
    return (v[l] == key) ? l : -1;
}

```

👉 usa il **prefetch** per anticipare il caricamento dei dati

👉 è utile quando il vero costo è la memoria



Riordinamento dell'array

```

int reorder_array(int * a, int * t, int len, int k, int i)
{
    if (k <= len) {
        i = reorder_array(a, t, len, 2 * k, i);
        t[k] = a[i];
        i += 1;
        i = reorder_array(a, t, len, 2 * k + 1, i);
    }
    return i;
}

```

👉 questo codice costruisce una nuova disposizione in memoria dell'array

👉 l'idea è memorizzare i valori come se fossero i nodi di un albero binario

✳️ così la ricerca sfrutta meglio la cache

Ricerca su array riordinato

```
int search_reordered(int * v, int len, int key)
{
    int i = 1;
    int pos = 0;
    while (i <= len)
    {
        pos = i * (v[i] == key);
        i = 2*i + (v[i] >= key);
    }
    return pos;
}
```

👉 questa versione cerca direttamente nell'array riordinato

👉 vantaggio:

- gli accessi seguono meglio la struttura ad albero
- la località di memoria migliora

Ricerca su array riordinato con prefetch

```
int search_reordered_prefetch(int * v, int len, int key)
{
    int i = 1;
    int pos = 0;
    while (i <= len)
    {
        pos = i * (v[i] == key);
        __builtin_prefetch(v + i * 16);
        __builtin_prefetch(v + i * 16 + 15);
        i = 2*i + (v[i] >= key);
    }
    return pos;
}
```

👉 combina:

- disposizione migliore in memoria
- prefetch

✳ è la versione più aggressiva dal punto di vista della cache

◆ Confronto finale

Possiamo quindi avere diverse strategie:

- con branch → sfrutta predizione e speculazione
- branchless → evita errori di predizione
- prefetch → anticipa l'accesso ai dati
- array riordinato → migliora la località in memoria

✳ la versione migliore dipende dal rapporto tra branch prediction e costo degli accessi alla memoria